Proyecto de Grado: IA al servicio de las comunidades

Maestría en Analítica para la Inteligencia de Negocios

Nombres: María Fernanda Izquierdo Aparicio & Ana María Ochoa Muñoz

Fecha: 19 de Mayo de 2025

Este modelo forma parte del proyecto de grado de la Maestría en Analítica para la Inteligencia de Negocios de la Pontificia Universidad Javeriana, enfocado en la aplicación de Inteligencia Artificial al servicio de las comunidades. Su objetivo es clasificar imágenes de plantas en función de su especie usando una arquitectura pre-entrenada UNet

1. Importe de librerias

In [3]:
```
import json

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from pathlib import Path

import os
import zipfile
import shutil
!pip install seaborn
import seaborn as sns
import datetime
!pip install opencv-python
import cv2


import keras
from keras import models
from keras import layers

import random

import tensorflow as tf
```

```
from tensorflow.keras.models import Model
from tensorflow.keras import Layer
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, Reduc
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Conv2D, MaxPooling2D, UpSampling2D, conc

!pip install scikit-learn

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

from PIL import Image  # Librería para abrir y manipular imágenes

# ignoring warnings
import warnings
warnings.simplefilter("ignore")
```

```
Requirement already satisfied: seaborn in /opt/anaconda3/lib/python3.12/site
-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /opt/anaconda3/lib/py
thon3.12/site-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in /opt/anaconda3/lib/python3.12/
site-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /opt/anaconda3/li
b/python3.12/site-packages (from seaborn) (3.9.2)
Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/lib/python
3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.2.0)
Requirement already satisfied: cycler>=0.10 in /opt/anaconda3/lib/python3.1
2/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/lib/pytho
n3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/lib/pytho
n3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.4)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/lib/python
3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.1)
Requirement already satisfied: pillow>=8 in /opt/anaconda3/lib/python3.12/si
te-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/lib/python
3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda3/lib/py
thon3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/lib/python3.1
2/site-packages (from pandas>=1.2->seaborn) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/lib/python3.
12/site-packages (from pandas>=1.2->seaborn) (2023.3)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.12/sit
e-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.
16.0)
Requirement already satisfied: opencv-python in /opt/anaconda3/lib/python3.1
2/site-packages (4.11.0.86)
Requirement already satisfied: numpy>=1.21.2 in /opt/anaconda3/lib/python3.1
2/site-packages (from opencv-python) (1.26.4)
Requirement already satisfied: scikit-learn in /opt/anaconda3/lib/python3.1
2/site-packages (1.5.1)
Requirement already satisfied: numpy>=1.19.5 in /opt/anaconda3/lib/python3.1
2/site-packages (from scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /opt/anaconda3/lib/python3.1
2/site-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /opt/anaconda3/lib/python3.1
2/site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/anaconda3/lib/py
thon3.12/site-packages (from scikit-learn) (3.5.0)
```

In [4]:
```python
#Medir_tiempo_ejecucion():
inicio = datetime.datetime.now()
print(f"Inicio: {inicio.strftime('%Y-%m-%d %H:%M:%S.%f')}")
```

```
Inicio: 2025-05-19 13:21:39.467764
```

2. Defínción de Rutas y Parámetros

In [6]:
```
data_dir = 'Datos Finales Estado Balanceados'  # Reemplaza con la ruta real
metadata_file = 'archivos_datos_balanceados_estado.csv' # Reemplaza con la r
image_height = 128
image_width = 128
batch_size = 32
num_classes = None  # Se determinará automáticamente del archivo CSV
```

3. Cargar y procesar los metadatos

In [9]:
```
try:
    metadata = pd.read_csv(metadata_file, sep=";")
    metadata = metadata.rename(columns={'Nombre del archivo': 'image_id'})
    metadata = metadata.rename(columns={'Planta_Estado': 'Planta_Estado_Espa

    # Asegúrate de que las columnas 'filename' (o el nombre de tu columna de
    # y 'label' (o el nombre de tu columna de etiquetas) existan.

    if 'image_id' not in metadata.columns or 'Planta_Estado_Español' not in
        raise ValueError("El archivo CSV debe contener las columnas 'image_i
    class_names = metadata['Planta_Estado_Español'].unique().tolist()
    num_classes = len(class_names)
    print(f"Se encontraron {num_classes} clases: {class_names}")
except FileNotFoundError:
    print(f"Error: No se encontró el archivo de metadatos en '{metadata_file
    exit()
except ValueError as e:
    print(f"Error al procesar el archivo CSV: {e}")
    exit()
```

```
Se encontraron 18 clases: ['Maíz_Enferma', 'Tomate_Saludable', 'Yuca_Enferm
a', 'Pepino Cohombro_Enferma', 'Tomate_Enferma', 'Manzana_Saludable', 'Fresa
_Enferma', 'Maíz_Saludable', 'Papa_Enferma', 'Fresa_Saludable', 'Coliflor_En
ferma', 'Pepino Cohombro_Saludable', 'Yuca_Saludable', 'Fríjol_Enferma', 'Pa
pa_Saludable', 'Fríjol_Saludable', 'Coliflor_Saludable', 'Manzana_Enferma']
```

In [13]:
```
metadata.Planta_Estado_Español.value_counts()
```

Out[13]:
```
Planta_Estado_Español
Fríjol_Saludable              3501
Maíz_Enferma                  3500
Tomate_Saludable              3500
Coliflor_Saludable            3500
Papa_Saludable                3500
Fríjol_Enferma                3500
Yuca_Saludable                3500
Pepino Cohombro_Saludable     3500
Coliflor_Enferma              3500
Fresa_Saludable               3500
Papa_Enferma                  3500
Maíz_Saludable                3500
Fresa_Enferma                 3500
Manzana_Saludable             3500
Tomate_Enferma                3500
Pepino Cohombro_Enferma       3500
Yuca_Enferma                  3500
Manzana_Enferma               3500
Name: count, dtype: int64
```

In [15]:
```
train, test= train_test_split(metadata, test_size=0.2, random_state=42,shuff
train.shape, test.shape
```

Out[15]:
```
((50400, 7), (12601, 7))
```

In [17]:
```
train.head()
```

Out[17]:

	Carpeta	image_id	Planta_Inglés	Estado_Ingles
50219	Potato_healthy	aug_966_683b04ad-7941-4819-9965-ba32c725eb22__...	Potato	healthy
12719	Cucumber_sick	aug_60_cucumber_photo_2020-06-27_23-51-02 (2).jpg	Cucumber	sick
23065	Strawberry_sick	aug_124_36bc7257-0b59-45a9-a692-1ff35b5a3425__...	Strawberry	sick
42154	Cassava_healthy	aug_799_919479322.jpg	Cassava	healthy
60493	Apple_sick	b1a2c1e8-e1a6-4d25-9d5b-1488e9d8b0ee___FREC_C....	Apple	sick

In [19]:
```
sample = train.sample(5)  # ejemplo común
```

In [21]:
```
print(sample.columns)
```

```
Index(['Carpeta', 'image_id', 'Planta_Inglés', 'Estado_Ingles',
       'Planta_Español', 'Estado_Español', 'Planta_Estado_Español'],
      dtype='object')
```

4. Crear un generador de datos

In [24]:
```python
# Crear el generador de imágenes
datagen = ImageDataGenerator(
    validation_split = 0.2,
    rescale=1./255
)

train_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
    y_col='Planta_Estado_Español',
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='categorical',  # Importante para clasificación multiclase
    subset='training',
    seed=42,
    classes=class_names  # Asegurarse del orden de las clases
)

validation_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
    y_col='Planta_Estado_Español',
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation',
    seed=42,
    classes=class_names
)
```

```
Found 40320 validated image filenames belonging to 18 classes.
Found 10079 validated image filenames belonging to 18 classes.
```

In [25]:
```python
print(train['Planta_Estado_Español'].value_counts())
print(f"Total clases: {train['Planta_Estado_Español'].nunique()}")
```

```
Planta_Estado_Español
Papa_Saludable               2800
Pepino Cohombro_Enferma      2800
Tomate_Saludable             2800
Papa_Enferma                 2800
Maíz_Saludable               2800
Fríjol_Enferma               2800
Coliflor_Enferma             2800
Pepino Cohombro_Saludable    2800
Fríjol_Saludable             2800
Manzana_Saludable            2800
Coliflor_Saludable           2800
Yuca_Enferma                 2800
Tomate_Enferma               2800
Fresa_Saludable              2800
Manzana_Enferma              2800
Yuca_Saludable               2800
Fresa_Enferma                2800
Maíz_Enferma                 2800
Name: count, dtype: int64
Total clases: 18
```

In [28]:
```python
from tensorflow.keras.preprocessing import image
import os
import numpy as np

img_path = os.path.join(data_dir, metadata["image_id"][20])
img = image.load_img(img_path, target_size=(image_height, image_width))
img_tensor = image.img_to_array(img)
img_tensor = np.expand_dims(img_tensor, axis=0)
img_tensor /= 255.

plt.imshow(img_tensor[0])
plt.axis('off')
plt.show()
```

5. Definición del modelo U-Net para clasificación

In [31]:
```
epochs=10
steps_per_epoch = len(train)*0.8 / batch_size
validation_steps = len(train)*0.2 / batch_size
```

In [33]:
```
# DEFINIR EL MODELO U-NET MODIFICADO PARA CLASIFICACIÓN
def unet_classification_model(input_shape=(image_height, image_width, 3), nu
    inputs = Input(shape=input_shape)

    # Encoder
    c1 = Conv2D(32, 3, activation='relu', padding='same')(inputs)
    c1 = Conv2D(32, 3, activation='relu', padding='same')(c1)
    p1 = MaxPooling2D()(c1)

    c2 = Conv2D(64, 3, activation='relu', padding='same')(p1)
    c2 = Conv2D(64, 3, activation='relu', padding='same')(c2)
    p2 = MaxPooling2D()(c2)

    c3 = Conv2D(128, 3, activation='relu', padding='same')(p2)
    c3 = Conv2D(128, 3, activation='relu', padding='same')(c3)
    p3 = MaxPooling2D()(c3)

    # Bottleneck
```

```
    c4 = Conv2D(256, 3, activation='relu', padding='same')(p3)
    c4 = Conv2D(256, 3, activation='relu', padding='same')(c4)

    # Decoder
    u5 = UpSampling2D()(c4)
    u5 = concatenate([u5, c3])
    c5 = Conv2D(128, 3, activation='relu', padding='same')(u5)
    c5 = Conv2D(128, 3, activation='relu', padding='same')(c5)

    u6 = UpSampling2D()(c5)
    u6 = concatenate([u6, c2])
    c6 = Conv2D(64, 3, activation='relu', padding='same')(u6)
    c6 = Conv2D(64, 3, activation='relu', padding='same')(c6)

    u7 = UpSampling2D()(c6)
    u7 = concatenate([u7, c1])
    c7 = Conv2D(32, 3, activation='relu', padding='same')(u7)
    c7 = Conv2D(32, 3, activation='relu', padding='same')(c7)

    # Clasificación
    gap = GlobalAveragePooling2D()(c7)
    outputs = Dense(num_classes, activation='softmax')(gap)

    model = Model(inputs=inputs, outputs=outputs)
    return model


# CREAR Y COMPILAR EL MODELO
model = unet_classification_model(input_shape=(image_height, image_width, 3)
model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['a
model.summary()  # Muestra la arquitectura
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 128, 128, 3)	0	–
conv2d (Conv2D)	(None, 128, 128, 32)	896	input_layer[0
conv2d_1 (Conv2D)	(None, 128, 128, 32)	9,248	conv2d[0][0]
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0	conv2d_1[0][0
conv2d_2 (Conv2D)	(None, 64, 64, 64)	18,496	max_pooling2d

conv2d_3 (Conv2D)	(None, 64, 64, 64)	36,928	conv2d_2[0][0
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0	conv2d_3[0][0
conv2d_4 (Conv2D)	(None, 32, 32, 128)	73,856	max_pooling2d
conv2d_5 (Conv2D)	(None, 32, 32, 128)	147,584	conv2d_4[0][0
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 128)	0	conv2d_5[0][0
conv2d_6 (Conv2D)	(None, 16, 16, 256)	295,168	max_pooling2d
conv2d_7 (Conv2D)	(None, 16, 16, 256)	590,080	conv2d_6[0][0
up_sampling2d (UpSampling2D)	(None, 32, 32, 256)	0	conv2d_7[0][0
concatenate (Concatenate)	(None, 32, 32, 384)	0	up_sampling2d conv2d_5[0][0
conv2d_8 (Conv2D)	(None, 32, 32, 128)	442,496	concatenate[0
conv2d_9 (Conv2D)	(None, 32, 32, 128)	147,584	conv2d_8[0][0
up_sampling2d_1 (UpSampling2D)	(None, 64, 64, 128)	0	conv2d_9[0][0
concatenate_1 (Concatenate)	(None, 64, 64, 192)	0	up_sampling2d conv2d_3[0][0
conv2d_10 (Conv2D)	(None, 64, 64, 64)	110,656	concatenate_1
conv2d_11 (Conv2D)	(None, 64, 64, 64)	36,928	conv2d_10[0][
up_sampling2d_2 (UpSampling2D)	(None, 128, 128, 64)	0	conv2d_11[0][
concatenate_2 (Concatenate)	(None, 128, 128, 96)	0	up_sampling2d conv2d_1[0][0
conv2d_12 (Conv2D)	(None, 128, 128, 32)	27,680	concatenate_2

conv2d_13 (Conv2D)	(None, 128, 128, 32)	9,248	conv2d_12[0][
global_average_poo… (GlobalAveragePool…	(None, 32)	0	conv2d_13[0][
dense (Dense)	(None, 18)	594	global_averag

Total params: 1,947,442 (7.43 MB)

Trainable params: 1,947,442 (7.43 MB)

Non-trainable params: 0 (0.00 B)

In [35]:
```
# Earlystopping
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',     # Monitorea la pérdida en validación
    patience=5,             # Número de épocas sin mejora antes de detener
    restore_best_weights=True,  # Restaura los mejores pesos
    verbose=1
)
```

6. Entrenamiento del Modelo

In [38]:
```
#Entrenamiento
history = model.fit(
    train_generator,
    validation_data=validation_generator,
    epochs=epochs,
    steps_per_epoch=int(steps_per_epoch),
    validation_steps=int(validation_steps),
    callbacks=[early_stop]
)
```

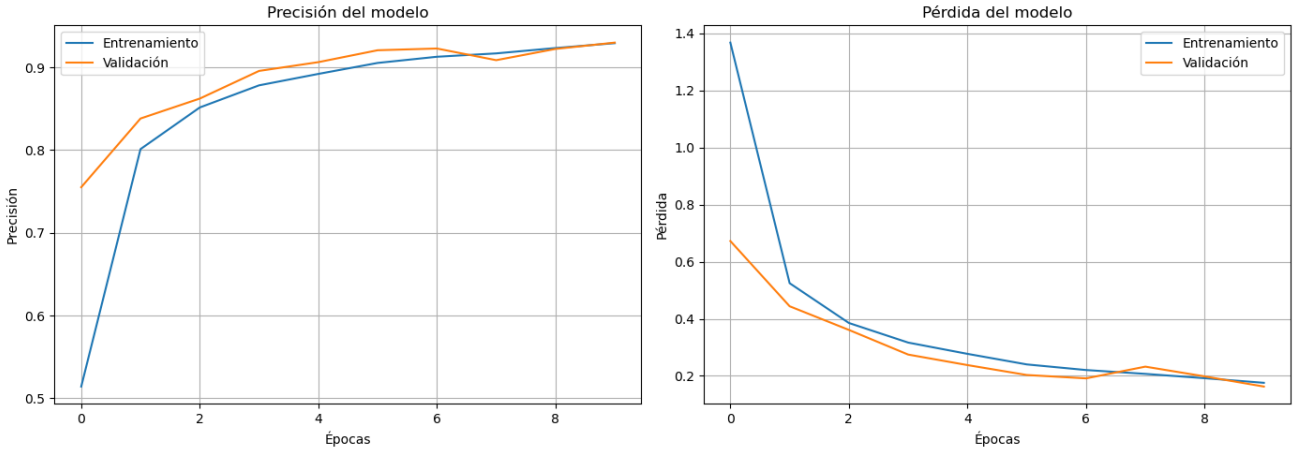
```
Epoch 1/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 3589s 3s/step - accuracy: 0.3297 - loss: 1.95
62 - val_accuracy: 0.7551 - val_loss: 0.6726
Epoch 2/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 3640s 3s/step - accuracy: 0.7728 - loss: 0.59
82 - val_accuracy: 0.8382 - val_loss: 0.4440
Epoch 3/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 3595s 3s/step - accuracy: 0.8446 - loss: 0.40
64 - val_accuracy: 0.8623 - val_loss: 0.3611
Epoch 4/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 3521s 3s/step - accuracy: 0.8786 - loss: 0.31
89 - val_accuracy: 0.8957 - val_loss: 0.2747
Epoch 5/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 3600s 3s/step - accuracy: 0.8903 - loss: 0.28
14 - val_accuracy: 0.9063 - val_loss: 0.2379
Epoch 6/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 4281s 3s/step - accuracy: 0.8996 - loss: 0.25
34 - val_accuracy: 0.9207 - val_loss: 0.2031
Epoch 7/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 3685s 3s/step - accuracy: 0.9117 - loss: 0.22
31 - val_accuracy: 0.9228 - val_loss: 0.1913
Epoch 8/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 3632s 3s/step - accuracy: 0.9158 - loss: 0.21
13 - val_accuracy: 0.9087 - val_loss: 0.2324
Epoch 9/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 3662s 3s/step - accuracy: 0.9245 - loss: 0.18
54 - val_accuracy: 0.9223 - val_loss: 0.1983
Epoch 10/10
1260/1260 ━━━━━━━━━━━━━━━━━━━━ 3658s 3s/step - accuracy: 0.9293 - loss: 0.17
32 - val_accuracy: 0.9300 - val_loss: 0.1626
Restoring model weights from the end of the best epoch: 10.
```

In [40]:
```
fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

```
Fin: 2025-05-20 06:46:19.122222
Duración: 17:24:39.654458
```

In [42]:
```
import matplotlib.pyplot as plt

# 1. Crear la figura
plt.figure(figsize=(14, 5))

# 2. Gráfico de Accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Entrenamiento')
plt.plot(history.history['val_accuracy'], label='Validación')
```

```
plt.title('Precisión del modelo')
plt.xlabel('Épocas')
plt.ylabel('Precisión')
plt.legend()
plt.grid(True)

# 3. Gráfico de Loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Entrenamiento')
plt.plot(history.history['val_loss'], label='Validación')
plt.title('Pérdida del modelo')
plt.xlabel('Épocas')
plt.ylabel('Pérdida')
plt.legend()
plt.grid(True)

# 4. Mostrar
plt.tight_layout()
plt.show()
```



In [44]:
```
print(test.columns.tolist())
```

```
['Carpeta', 'image_id', 'Planta_Inglés', 'Estado_Ingles', 'Planta_Español',
'Estado_Español', 'Planta_Estado_Español']
```

In [46]:
```
epochs = list(range(1, 11))

loss = history.history['loss']
val_loss = history.history['val_loss']
accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']


data = {
    'Epoch': epochs,
    'Trainning Loss': loss,
    'Validation Loss': val_loss,
```

```
    'Training Accuracy': accuracy,
    'Validation  Accuracy': val_accuracy
}
df = pd.DataFrame(data)

# Imprimir la tabla
print(df)
```

```
   Epoch  Trainning Loss  Validation Loss  Training Accuracy  \
0      1        1.367985         0.672587           0.514087
1      2        0.524849         0.444013           0.801190
2      3        0.385347         0.361069           0.851538
3      4        0.316721         0.274693           0.878299
4      5        0.277235         0.237871           0.892113
5      6        0.240198         0.203128           0.905382
6      7        0.220536         0.191302           0.912847
7      8        0.207099         0.232409           0.916915
8      9        0.191916         0.198269           0.923388
9     10        0.175775         0.162627           0.929291

   Validation  Accuracy
0             0.755134
1             0.838178
2             0.862288
3             0.895724
4             0.906340
5             0.920726
6             0.922810
7             0.908721
8             0.922314
9             0.929953
```

In [48]:
```
print('Perdida final en Train:', history.history['loss'][-1])
print("Pérdida final en Validation:", history.history['val_loss'][-1])

print("Accuracy final en Train:", history.history['accuracy'][-1])
print("Accuracy final en Validation:", history.history['val_accuracy'][-1])
```

```
Perdida final en Train: 0.17577514052391052
Pérdida final en Validation: 0.16262662410736084
Accuracy final en Train: 0.9292906522750854
Accuracy final en Validation: 0.929953396320343
```

In [64]:
```
#validación con datos de test

test_generator =  datagen.flow_from_dataframe(test,
                          directory = data_dir,
                          x_col = "image_id",
                          y_col = "Planta_Estado_Español",
                          target_size = (image_height,image_width),
                          batch_size = batch_size,
```

```
                                    class_mode = "categorical")
```

```
Found 12601 validated image filenames belonging to 18 classes.
```

7. Testeo del Modelo

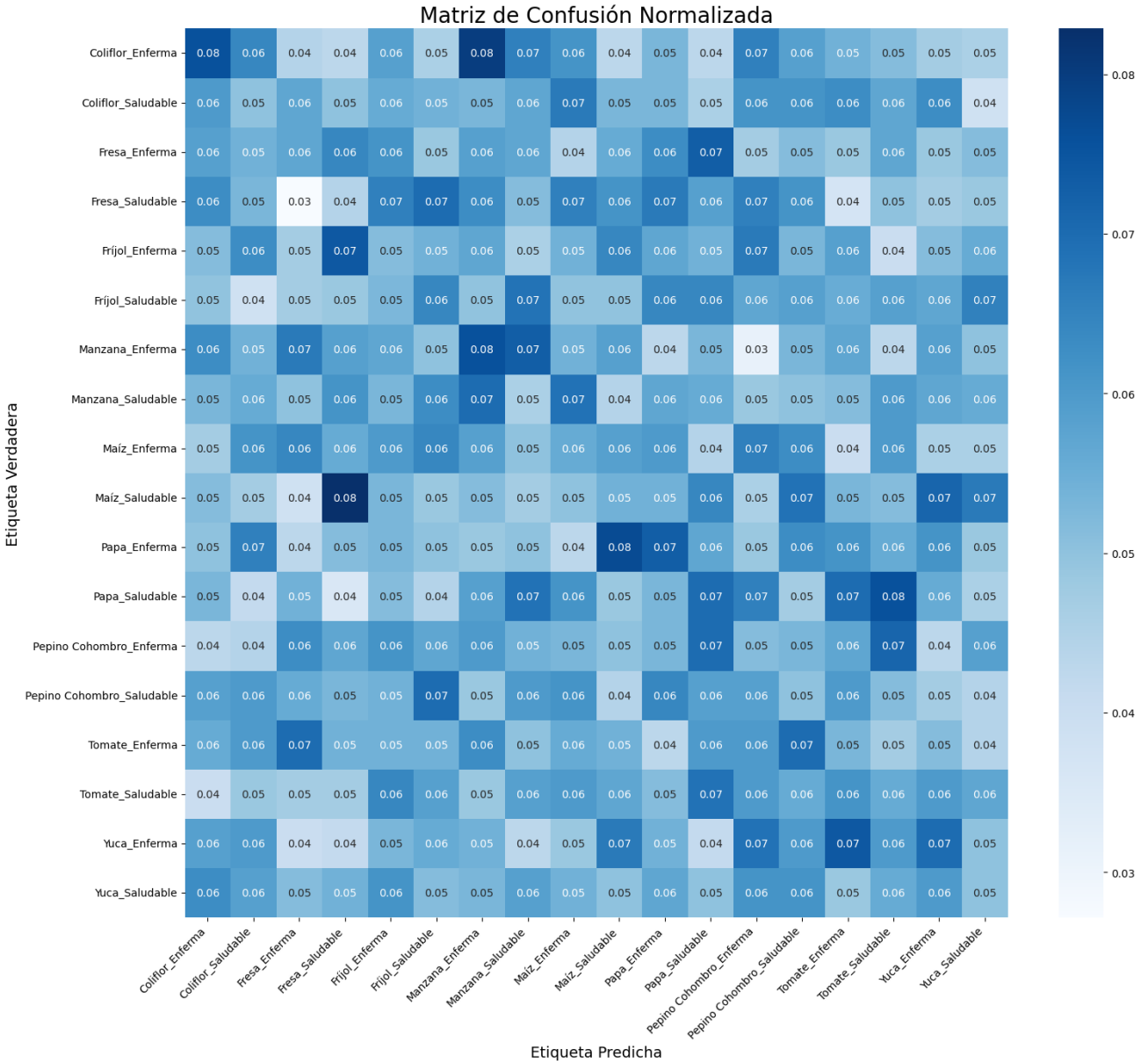
In [67]:
```
# Evaluar sobre el set de test
test_loss, test_accuracy = model.evaluate(test_generator, steps=len(test_ger

print(f'🔎 Pérdida en test: {test_loss:.4f}')
print(f'✅ Precisión en test: {test_accuracy:.2%}')
```

```
394/394 ━━━━━━━━━━━━━━━━━━━━ 278s 706ms/step - accuracy: 0.0018 - loss: 21.7
802
🔎 Pérdida en test: 21.7191
✅ Precisión en test: 0.13%
```

In [68]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# 1. Predecir clases sobre el test set
# Obtener las predicciones del modelo
y_pred_probs = model.predict(test_generator, steps=len(test_generator), verb

# Convertir probabilidades a etiquetas (argmax)
y_pred = np.argmax(y_pred_probs, axis=1)

# 2. Obtener etiquetas verdaderas
# Las verdaderas etiquetas están en test_generator.classes
y_true = test_generator.classes

# 3. Obtener los nombres de las clases
class_labels = list(test_generator.class_indices.keys())

# 4. Calcular la matriz de confusión
cm = confusion_matrix(y_true, y_pred)

# 5. Normalizar la matriz de confusión (opcional para mejor visualización)
cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

# 6. Graficar la matriz de confusión
plt.figure(figsize=(16, 14))
sns.heatmap(cm_normalized, annot=True, fmt='.2f', cmap='Blues', xticklabels=
plt.title('Matriz de Confusión Normalizada', fontsize=20)
plt.ylabel('Etiqueta Verdadera', fontsize=14)
plt.xlabel('Etiqueta Predicha', fontsize=14)
```

```
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

# 7. Reporte de clasificación (precisión, recall, f1-score por clase)
print('\nReporte de Clasificación:')
print(classification_report(y_true, y_pred, target_names=class_labels, digit
```

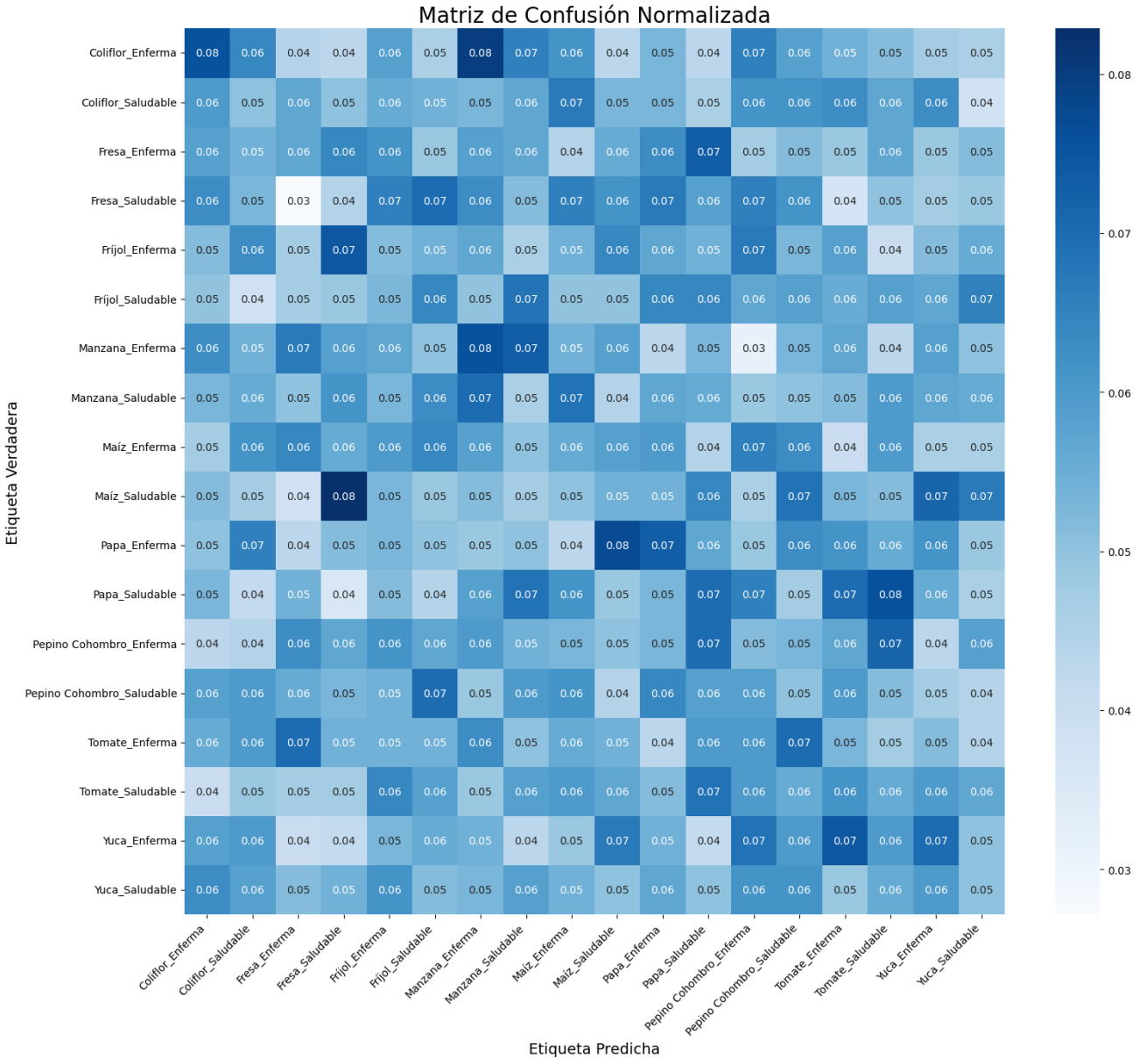
```
394/394 ━━━━━━━━━━━━━━━━━━━━ 295s 749ms/step
```



```
Reporte de Clasificación:
                           precision    recall  f1-score   support

          Coliflor_Enferma     0.0761    0.0757    0.0759       700
       Coliflor_Saludable     0.0510    0.0500    0.0505       700
            Fresa_Enferma     0.0618    0.0571    0.0594       700
         Fresa_Saludable      0.0453    0.0443    0.0448       700
          Fríjol_Enferma      0.0503    0.0514    0.0509       700
       Fríjol_Saludable      0.0640    0.0642    0.0641       701
         Manzana_Enferma     0.0720    0.0757    0.0738       700
       Manzana_Saludable     0.0455    0.0457    0.0456       700
            Maíz_Enferma     0.0552    0.0557    0.0555       700
         Maíz_Saludable      0.0548    0.0543    0.0545       700
           Papa_Enferma      0.0714    0.0729    0.0721       700
         Papa_Saludable      0.0679    0.0700    0.0689       700
  Pepino Cohombro_Enferma    0.0499    0.0514    0.0507       700
Pepino Cohombro_Saludable    0.0481    0.0500    0.0491       700
          Tomate_Enferma     0.0526    0.0529    0.0527       700
        Tomate_Saludable     0.0556    0.0557    0.0557       700
            Yuca_Enferma     0.0703    0.0700    0.0702       700
         Yuca_Saludable      0.0542    0.0500    0.0520       700

                accuracy                         0.0582     12601
               macro avg     0.0581    0.0582    0.0581     12601
            weighted avg     0.0581    0.0582    0.0581     12601
```

In [69]:

```python
# 1. Guardar la figura de la matriz de confusión
plt.figure(figsize=(16, 14))
sns.heatmap(cm_normalized, annot=True, fmt='.2f', cmap='Blues', xticklabels=
plt.title('Matriz de Confusión Normalizada', fontsize=20)
plt.ylabel('Etiqueta Verdadera', fontsize=14)
plt.xlabel('Etiqueta Predicha', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()

# Guardar el gráfico como imagen
plt.savefig(r'C:\Proyecto de grado\Modelos\matriz_confusion.png', dpi=300)
plt.show()
```



In [70]:
```
# 2. Guardar el reporte de clasificación en CSV

# Convertir el reporte a un diccionario
from sklearn.metrics import classification_report

report = classification_report(y_true, y_pred, target_names=class_labels, ou

# Convertirlo a DataFrame
report_df = pd.DataFrame(report).transpose()

# Guardar como CSV
report_df.to_csv(r'C:\Proyecto de grado\Modelos\reporte_clasificacion.csv',

print("✅ Reporte de clasificación guardado en C:\\Proyecto de grado\\Modelo
```

✅ Reporte de clasificación guardado en C:\Proyecto de grado\Modelos\reporte_clasificacion.csv

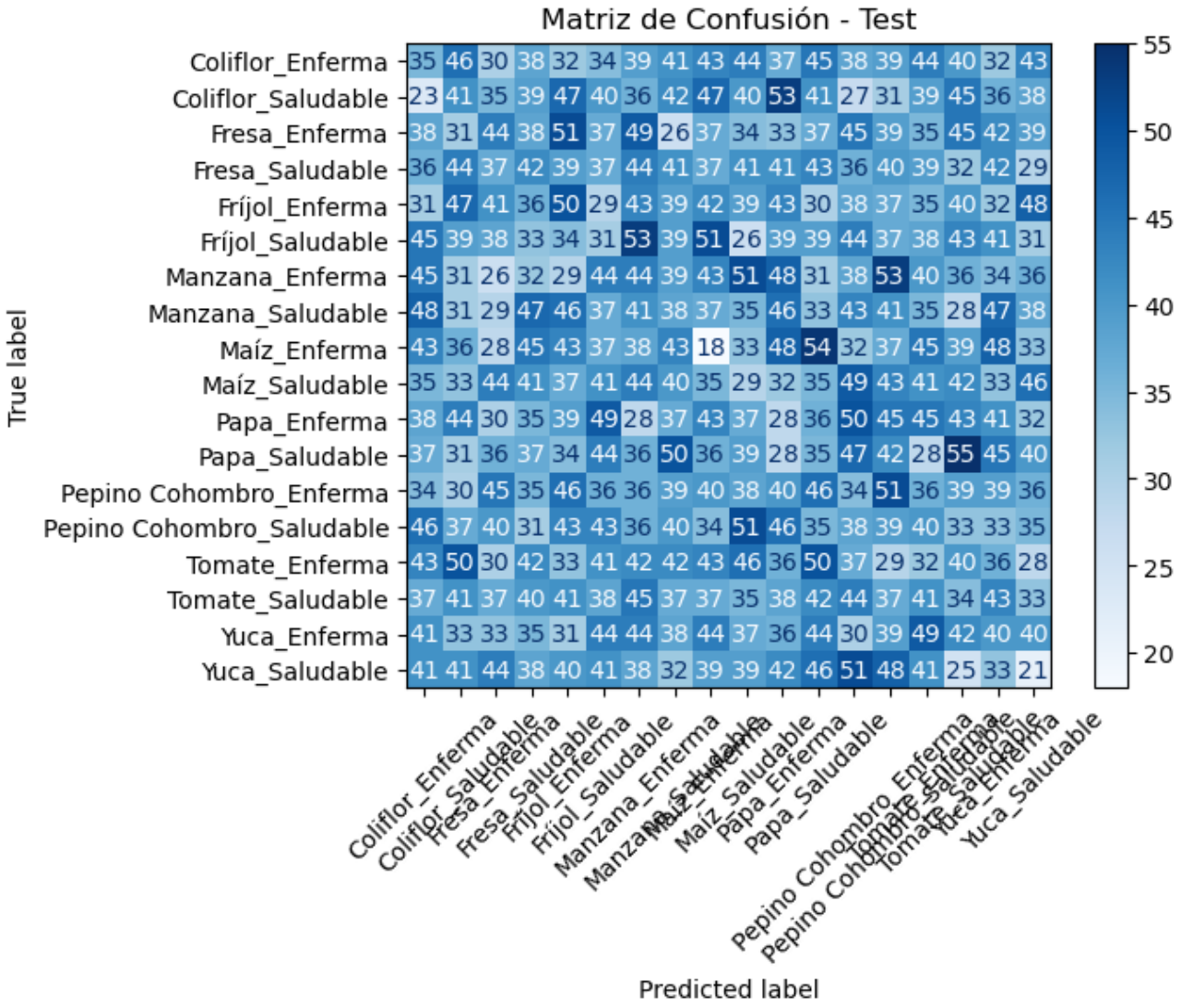
In [71]:
```
accuracy = accuracy_score(y_true, y_pred)
print(f"Test Accuracy: {accuracy:.4f}")
```

```
Test Accuracy: 0.0582
```

In [72]:
```
# ------------------------
# 1. Obtener predicciones completas para test
# ------------------------
y_pred_probs = model.predict(test_generator)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = test_generator.classes
```

```
394/394 ━━━━━━━━━━━━━━━━━━━━ 305s 775ms/step
```

In [73]:
```
# ------------------------
# 2. Matriz de Confusión
# ------------------------
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatri

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=test_gener
disp.plot(cmap="Blues", xticks_rotation=45)
plt.title("Matriz de Confusión - Test")
plt.show()
```



In [74]:
```python
# Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())

# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
```

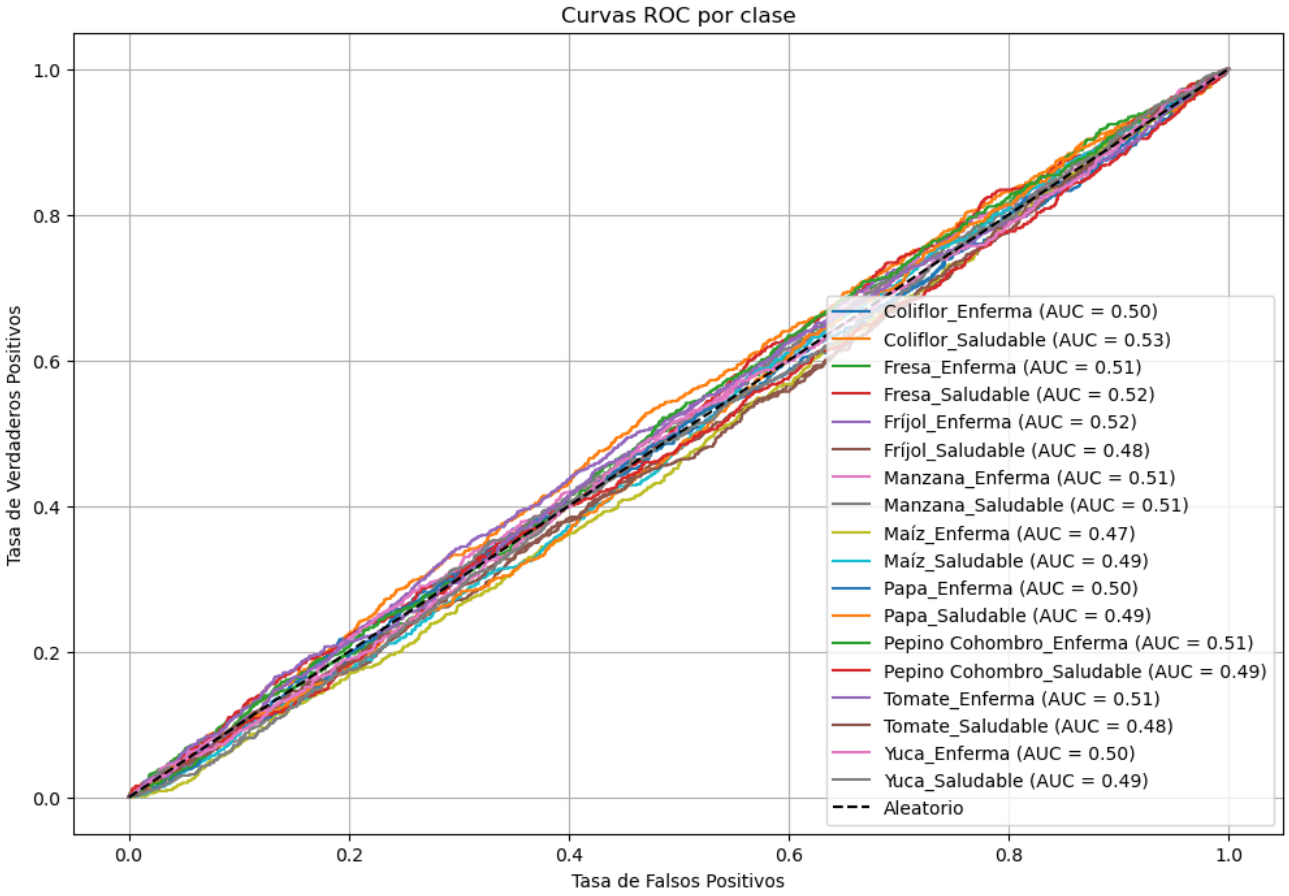
```
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
y_true_batch = np.argmax(y_true_batch, axis=1)

# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_ba
        color='green' if y_true_batch[i] == y_pred_batch[i] else 'red'
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 851ms/step
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[74], line 28
     26 plt.figure(figsize=(15, 8))
     27 for i in range(num_samples):
---> 28     plt.subplot(3, 4, i + 1)
     29     img_to_show = normalize_for_display(x_batch[i])
     30     plt.imshow(img_to_show)

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/pyplot.py:1534,
in subplot(*args, **kwargs)
   1531 fig = gcf()
   1533 # First, search for an existing subplot with a matching spec.
-> 1534 key = SubplotSpec._from_subplot_args(fig, args)
   1536 for ax in fig.axes:
   1537     # If we found an Axes at the position, we can reuse it if the us
er passed no
   1538     # kwargs or if the Axes class and kwargs are identical.
   1539     if (ax.get_subplotspec() == key
   1540         and (kwargs == {}
   1541              or (ax._projection_init
   1542                  == fig._process_projection_requirements(**kwargs)))
):

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/gridspec.py:589,
in SubplotSpec._from_subplot_args(figure, args)
    587 else:
    588     if not isinstance(num, Integral) or num < 1 or num > rows*cols:
--> 589         raise ValueError(
    590             f"num must be an integer with 1 <= num <= {rows*cols}, "
    591             f"not {num!r}"
    592         )
    593     i = j = num
    594 return gs[i-1:j]

ValueError: num must be an integer with 1 <= num <= 12, not 13
```

In [82]:
```
from sklearn.metrics import roc_curve, auc
from tensorflow.keras.utils import to_categorical

# 3. Convertir etiquetas reales a one-hot
n_classes = y_pred_probs.shape[1]
y_true_onehot = to_categorical(y_true, num_classes=n_classes)

# 4. Calcular ROC y AUC por clase
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_onehot[:, i], y_pred_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# 5. Obtener nombres de clase
class_names = list(test_generator.class_indices.keys())

# 6. Graficar curvas ROC
plt.figure(figsize=(12, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f

plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
```

```
plt.title('Curvas ROC por clase')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```



In [84]:
```
from sklearn.metrics import roc_auc_score

# AUC macro (promedio de clases, todas igual de importantes)
auc_macro = roc_auc_score(y_true_onehot, y_pred_probs, average='macro')

# AUC micro (promedia sobre todas las instancias)
auc_micro = roc_auc_score(y_true_onehot, y_pred_probs, average='micro')

print(f"AUC Macro promedio: {auc_macro:.4f}")
print(f"AUC Micro promedio: {auc_micro:.4f}")
```

```
AUC Macro promedio: 0.5003
AUC Micro promedio: 0.4997
```

In [86]:
```
# Asegúrate de tener esto:
# y_pred = np.argmax(y_pred_probs, axis=1)
# y_true = test_generator.classes

# Accuracy
accuracy = accuracy_score(y_true, y_pred)
```

```
# F1 Scores
f1_macro = f1_score(y_true, y_pred, average='macro')
f1_micro = f1_score(y_true, y_pred, average='micro')
f1_weighted = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score (macro): {f1_macro:.4f}")
print(f"F1 Score (micro): {f1_micro:.4f}")
print(f"F1 Score (weighted): {f1_weighted:.4f}")

# Detalle por clase
class_names = list(test_generator.class_indices.keys())
print("\nReporte de clasificación por clase:")
print(classification_report(y_true, y_pred, target_names=class_names))
```

```
Accuracy: 0.0504
F1 Score (macro): 0.0504
F1 Score (micro): 0.0504
F1 Score (weighted): 0.0504

Reporte de clasificación por clase:
                           precision    recall  f1-score   support

         Coliflor_Enferma       0.05      0.05      0.05       700
       Coliflor_Saludable       0.06      0.06      0.06       700
            Fresa_Enferma       0.07      0.06      0.07       700
          Fresa_Saludable       0.06      0.06      0.06       700
           Fríjol_Enferma       0.07      0.07      0.07       700
         Fríjol_Saludable       0.04      0.04      0.04       701
          Manzana_Enferma       0.06      0.06      0.06       700
        Manzana_Saludable       0.05      0.05      0.05       700
             Maíz_Enferma       0.03      0.03      0.03       700
           Maíz_Saludable       0.04      0.04      0.04       700
            Papa_Enferma       0.04      0.04      0.04       700
          Papa_Saludable       0.05      0.05      0.05       700
  Pepino Cohombro_Enferma       0.05      0.05      0.05       700
Pepino Cohombro_Saludable       0.05      0.06      0.05       700
           Tomate_Enferma       0.05      0.05      0.05       700
         Tomate_Saludable       0.05      0.05      0.05       700
             Yuca_Enferma       0.06      0.06      0.06       700
           Yuca_Saludable       0.03      0.03      0.03       700

                 accuracy                           0.05     12601
                macro avg       0.05      0.05      0.05     12601
             weighted avg       0.05      0.05      0.05     12601
```

8. Guardar el modelo entrenado

In [89]:
```
model.save('modelo_16_Unet_PlantaEstado_Balanceado.h5')
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
```

In [91]:
```
fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

```
Fin: 2025-05-20 07:06:44.344503
Duración: 17:45:04.876739
```

In [93]:
```
class_indices = train_generator.class_indices  # o test_generator.class_indi
index_to_class = {v: k for k, v in class_indices.items()}
print(index_to_class)
with open("index_to_class_modelo16_Balanceado.json", "w") as f:
    json.dump(index_to_class, f, indent=4, ensure_ascii=False)
```

```
{0: 'Maíz_Enferma', 1: 'Tomate_Saludable', 2: 'Yuca_Enferma', 3: 'Pepino Cohombro_Enferma', 4: 'Tomate_Enferma', 5: 'Manzana_Saludable', 6: 'Fresa_Enferma', 7: 'Maíz_Saludable', 8: 'Papa_Enferma', 9: 'Fresa_Saludable', 10: 'Coliflor_Enferma', 11: 'Pepino Cohombro_Saludable', 12: 'Yuca_Saludable', 13: 'Fríjol_Enferma', 14: 'Papa_Saludable', 15: 'Fríjol_Saludable', 16: 'Coliflor_Saludable', 17: 'Manzana_Enferma'}
```

In [ ]:

In [ ]:

In [ ]: